

Tutorial de Aula

Python Fundamental + NumPy + Pandas

Listas • Dicionários • Funções • NumPy • Pandas

Objetivos de Aprendizagem

Ao final desta aula, o aluno será capaz de:

- Criar, acessar e manipular listas em Python.
- Criar e operar dicionários para armazenar dados estruturados.
- Definir e chamar funções com parâmetros, retorno e valores padrão.
- Importar e utilizar a biblioteca NumPy para operações com arrays.
- Criar e manipular DataFrames com Pandas para análise de dados tabulares.
- Aplicar esses conceitos em exemplos práticos do cotidiano.

MÓDULO 1 — LISTAS

1. Listas (lists)

Uma lista é uma estrutura de dados que armazena uma sequência ordenada de elementos. Os elementos podem ser de qualquer tipo — números, textos, booleanos ou até outras listas.

1.1 Criando e Acessando Listas

Sintaxe básica: os elementos são separados por vírgulas e delimitados por colchetes.

```
# Criando listas
frutas = ["maçã", "banana", "laranja", "uva"]
numeros = [10, 25, 37, 42, 58]
mista = [1, "Python", True, 3.14]

# Acessando pelo índice (inicia em 0)
print(frutas[0]) # maçã

# Fatiamento (slicing) [início : fim : passo]
print(frutas[1:3]) # ["banana", "laranja"]
print(numeros[::2])# [10, 37, 58]
```

 **Índice negativo:** lista[-1] retorna o último elemento; lista[-2] o penúltimo, e assim por diante.

1.2 Principais Métodos de Listas

Os métodos abaixo são os mais utilizados no dia a dia:

```
lista = [3, 1, 4, 1, 5, 9]

lista.append(2)      # adiciona 2 ao final
lista.insert(0, 0)   # insere 0 na posição 0
lista.remove(1)      # remove a primeira ocorrência do valor 1
lista.pop()          # remove e retorna o último elemento
lista.pop(2)         # remove e retorna o índice 2
lista.sort()         # ordena in-place (menor → maior)
lista.reverse()      # inverte a ordem in-place
lista.index(5)       # retorna o índice de 5
lista.count(1)       # conta quantas vezes 1 aparece
len(lista)           # comprimento da lista
```

1.3 Iterando sobre Listas

```
# for simples
for fruta in frutas:
    print(fruta)

# enumerate — índice + valor
for i, fruta in enumerate(frutas):
    print(f"{i}: {fruta}")

# list comprehension — cria lista nova
quadrados = [x**2 for x in range(1, 6)]
# [1, 4, 9, 16, 25]

# com filtro
pares = [x for x in range(10) if x % 2 == 0]
```

```
# [0, 2, 4, 6, 8]
```

1.4 Exemplo Prático — Gerenciador de Notas

A seguir, um mini-programa que cadastra notas de alunos e calcula a média da turma:

```
notas = []

# Entrada de dados
for i in range(5):
    nota = float(input(f"Digite a nota do aluno {i+1}: "))
    notas.append(nota)

# Cálculos
media = sum(notas) / len(notas)
maior = max(notas)
menor = min(notas)
aprovados = [n for n in notas if n >= 6.0]

print(f"Notas:      {notas}")
print(f"Média:      {media:.2f}")
print(f"Maior nota: {maior}")
print(f"Menor nota: {menor}")
print(f"Aprovados:  {len(aprovados)} de {len(notas)}")
```

2. Dicionários (dicts)

Um dicionário armazena pares chave → valor. As chaves devem ser únicas e imutáveis (str, int, tuple); os valores podem ser de qualquer tipo.

2.1 Criando e Acessando Dicionários

```
# Criação literal
aluno = {
    "nome": "Ana Paula",
    "idade": 21,
    "curso": "Ciência da Computação",
    "notas": [8.5, 9.0, 7.5],
}

# Acesso
print(aluno["nome"])          # Ana Paula
print(aluno.get("idade"))     # 21
print(aluno.get("email", "-")) # - (padrão se ausente)

# Adicionando / atualizando
aluno["email"] = "ana@uni.br"
aluno["idade"] = 22

# Removendo
del aluno["email"]
matricula = aluno.pop("matricula", None) # seguro
```

2.2 Métodos Essenciais

```
peessoa = {"nome": "Carlos", "cidade": "Fortaleza", "idade": 30}

peessoa.keys()    # dict_keys(["nome", "cidade", "idade"])
peessoa.values()  # dict_values(["Carlos", "Fortaleza", 30])
peessoa.items()   # dict_items([("nome", "Carlos"), ...])

# Verificar existência
"nome" in peessoa # True
"email" in peessoa # False

# Mesclar dicionários
extra = {"email": "carlos@mail.com"}
completo = peessoa | extra
```

2.3 Iterando sobre Dicionários

```
# Percorrendo pares chave-valor
for chave, valor in aluno.items():
    print(f"{chave}: {valor}")

quadrados = {x: x**2 for x in range(1, 6)}
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Filtrar por condição
```

```
maiores = {k: v for k, v in quadrados.items() if v > 9}
# {4: 16, 5: 25}
```

2.4 Exemplo Prático — Catálogo de Produtos

```
catalogo = {
    "notebook": {"preco": 3200.00, "estoque": 15},
    "mouse":    {"preco": 89.90, "estoque": 40},
    "teclado":  {"preco": 150.00, "estoque": 8},
}

def exibir_catalago(cat):
    print(f{"Produto":<12} {"Preço":>10} {"Estoque":>9}")
    print("-" * 34)
    for produto, info in cat.items():
        print(f{"produto":<12} R${info["preco"]:>8.2f} {info["estoque"]:>9}")

exibir_catalago(catalogo)

# Total em estoque
total = sum(i["preco"] * i["estoque"] for i in catalogo.values())
print(f"\nValor total em estoque: R$ {total:,.2f}")
```

MÓDULO 3 — FUNÇÕES

3. Funções

Funções permitem encapsular um bloco de código reutilizável. Em Python elas são definidas com a palavra-chave

```
def saudacao(nome):  
    """Exibe uma mensagem de boas-vindas."""  
    print(f"Olá, {nome}! Bem-vindo ao Python.")  
  
saudacao("Maria") # Olá, Maria! Bem-vindo ao Python.
```



Docstring: Sempre documente suas funções com uma string entre aspas triplas logo após o def. Isso facilita a manutenção e gera documentação automática com help().

3.1 Parâmetros e Retorno

```
# Parâmetros com valor padrão  
def potencia(base, expoente=2):  
    return base ** expoente  
  
print(potencia(3)) # 9 (expoente padrão)  
print(potencia(2, 10)) # 1024  
  
# Retorno múltiplo (tupla)  
def estatisticas(nums):  
    return min(nums), max(nums), sum(nums) / len(nums)  
  
mi, ma, med = estatisticas([4, 7, 2, 9, 1])  
print(f"Min={mi} Max={ma} Média={med}")
```

3.2 *args e **kwargs

```
# *args - número variável de argumentos posicionais  
def somar(*numeros):  
    return sum(numeros)  
  
print(somar(1, 2, 3)) # 6  
print(somar(10, 20, 30, 40)) # 100  
  
# **kwargs - argumentos nomeados arbitrários  
def apresentar(**dados):  
    for chave, valor in dados.items():  
        print(f" {chave}: {valor}")  
  
apresentar(nome="Lucas", cidade="Recife", profissao="Dev")
```

3.3 Funções de Alta Ordem e Lambda

```
# sorted() com chave personalizada  
alunos = [("Ana", 8.5), ("Bruno", 7.0), ("Carla", 9.2)]  
ranking = sorted(alunos, key=lambda a: a[1], reverse=True)  
# [("Carla", 9.2), ("Ana", 8.5), ("Bruno", 7.0)]  
  
# map() - aplica função a cada elemento
```

```
notas = [6.0, 7.5, 8.0, 5.5]
arredondadas = list(map(lambda n: round(n), notas))

# filter() – filtra elementos
aprovados = list(filter(lambda n: n >= 6, notas))
```

3.4 Exemplo Prático — Calculadora de IMC

```
def calcular_imc(peso, altura):
    """Calcula o Índice de Massa Corporal."""
    return peso / altura ** 2

def classificar_imc(imc):
    """Retorna a classificação do IMC."""
    if imc < 18.5:
        return "Abaixo do peso"
    elif imc < 25.0:
        return "Peso normal"
    elif imc < 30.0:
        return "Sobrepeso"
    else:
        return "Obesidade"

def relatorio_imc(nome, peso, altura):
    """Gera relatório completo de IMC."""
    imc = calcular_imc(peso, altura)
    classe = classificar_imc(imc)
    print(f"Paciente : {nome}")
    print(f"IMC      : {imc:.2f}")
    print(f"Situação  : {classe}")

relatorio_imc("João Silva", 82, 1.75)
```

4. Introdução ao NumPy

NumPy (Numerical Python) é a biblioteca essencial para computação científica em Python. Ela fornece o objeto `ndarray` — um array multidimensional eficiente — e funções matemáticas de alto desempenho.

4.1 Instalação e Importação

```
# Instalação (caso necessário)
pip install numpy

# Importação padrão
import numpy as np

print(np.__version__) # ex.: 1.26.4
```

4.2 Criando Arrays NumPy

```
import numpy as np

# A partir de listas Python
v = np.array([1, 2, 3, 4, 5])      # 1-D
M = np.array([[1, 2, 3], [4, 5, 6]]) # 2-D (matriz)

# Arrays especiais
zeros = np.zeros((3, 4)) # zeros 3x4
uns = np.ones((2, 3)) # uns 2x3
ident = np.eye(3) # identidade 3x3
range_ = np.arange(0, 10, 2) # [0 2 4 6 8]
linsp = np.linspace(0, 1, 5) # [0.00 0.25 0.50 0.75 1.00]
aleats = np.random.rand(3, 3) # 3x3 com valores aleatórios em [0, 1)

# Informações
print(M.shape) # (2, 3)
print(M.ndim) # 2
print(M.dtype) # int64
print(M.size) # 6
```

4.3 Indexação e Fatiamento

```
A = np.array([[10, 20, 30],
               [40, 50, 60],
               [70, 80, 90]])

# Elemento individual
print(A[0, 0]) # 10
print(A[2, 1]) # 80

# Linha / coluna completa
print(A[1, :]) # [40 50 60]
print(A[:, 2]) # [30 60 90]

# Sub-matriz
print(A[:2, 1:]) # [[20 30] [50 60]]
```

```
# Indexação booleana
print(A[A > 50]) # [60 70 80 90]
A[A < 30] = 0    # zero elementos menores que 30
```

4.4 Operações Matemáticas

O NumPy aplica operações elemento a elemento automaticamente (broadcasting):

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Operações elemento a elemento
print(a + b) # [5 7 9]
print(a * b) # [4 10 18]
print(a ** 2) # [1 4 9]
print(a + 10) # [11 12 13] - broadcasting escalar

# Funções matemáticas
print(np.sqrt(a)) # [1. 1.41 1.73]
print(np.exp(a)) # [2.72 7.39 20.09]
print(np.log(b)) # [1.39 1.61 1.79]

# Produto matricial
M1 = np.array([[1, 2], [3, 4]])
M2 = np.array([[5, 6], [7, 8]])
print(M1 @ M2) # [[19 22] [43 50]]
```

4.5 Estatísticas e Agregações

```
dados = np.array([23, 45, 12, 67, 34, 89, 56, 11, 78, 43])

print("Média :", np.mean(dados)) # 45.8
print("Mediana:", np.median(dados)) # 44.0
print("Desvio :", np.std(dados)) # 25.85
print("Mínimo :", np.min(dados)) # 11
print("Máximo :", np.max(dados)) # 89
print("Soma :", np.sum(dados)) # 458

# Ao longo de eixos (2-D)
M = np.array([[1, 2, 3], [4, 5, 6]])
print(np.sum(M, axis=0)) # [5 7 9] - soma por coluna
print(np.sum(M, axis=1)) # [6 15] - soma por linha
```

4.6 Reshape e Transformações

```
v = np.arange(1, 13) # [ 1  2  3 ... 12]
M = v.reshape(3, 4) # matriz 3x4
M2 = v.reshape(2, 2, 3) # tensor 2x2x3

# Transposição
print(M.T.shape) # (4, 3)

# Aplanar para 1-D
print(M.flatten()) # [ 1  2  3 ... 12]

# Concatenar
a = np.ones((2, 3))
b = np.zeros((2, 3))
print(np.vstack([a, b]).shape) # (4, 3)
print(np.hstack([a, b]).shape) # (2, 6)
```

4.7 Exemplo Prático — Análise de Vendas Mensais

Problema: analisar as vendas de três produtos ao longo de seis meses.

```
import numpy as np

# Linhas: produtos (Notebook, Mouse, Teclado)
# Colunas: meses (Jan a Jun)
vendas = np.array([
    [120, 135, 148, 162, 175, 190],
    [340, 298, 312, 400, 390, 420],
    [ 88,  95, 102, 115,  97, 108],
])

produtos = ["Notebook", "Mouse", "Teclado"]
meses     = ["Jan", "Fev", "Mar", "Abr", "Mai", "Jun"]

# Total por produto (soma em linha)
total_produto = vendas.sum(axis=1)
for prod, total in zip(produtos, total_produto):
    print(f"{prod:<10}: {total} unidades")

# Média mensal geral
media_mes = vendas.mean(axis=0)
for mes, med in zip(meses, media_mes):
    print(f"{mes}: {med:.1f} unidades/produto")

# Melhor mês (maior total geral)
total_mes = vendas.sum(axis=0)
melhor_mes = meses[np.argmax(total_mes)]
print(f"\nMelhor mês: {melhor_mes} ({total_mes.max()} unidades)")

# Percentual de crescimento Jan → Jun
crescimento = (vendas[:, -1] - vendas[:, 0]) / vendas[:, 0] * 100
for prod, cresc in zip(produtos, crescimento):
    print(f"{prod:<10}: {cresc:+.1f}%")
```

5. Introdução ao Pandas

Pandas é a principal biblioteca Python para análise e manipulação de dados tabulares. Ela fornece duas estruturas centrais: a Series (array 1-D rotulado) e o DataFrame (tabela 2-D com linhas e colunas nomeadas), construídas sobre o NumPy.

5.1 Instalação e Importação

```
# Instalação
pip install pandas

# Importação padrão
import pandas as pd
import numpy as np    # frequentemente usado junto

print(pd.__version__) # ex.: 2.2.2
```

5.2 Series — Array 1-D Rotulado

Uma Series é como uma coluna de planilha: possui valores e um índice (rótulo) para cada posição.

```
# Criando Series
notas = pd.Series([8.5, 7.0, 9.2, 6.3],
                  index=["Ana", "Bruno", "Carla", "Diego"])

# Acesso por rótulo e por posição
print(notas["Ana"])    # 8.5
print(notas.iloc[2])   # 9.2 (posição inteira)

# Operações vetorizadas
print(notas * 10)      # multiplica todos por 10
print(notas[notas >= 7.0]) # filtra aprovados

# Estatísticas rápidas
print(notas.mean())    # média
print(notas.describe()) # resumo estatístico completo
```



Índice personalizado: O índice de uma Series pode ser texto, datas ou qualquer tipo imutável — isso facilita buscas e joins sem precisar lembrar a posição numérica.

5.3 DataFrame — Tabela 2-D

Um DataFrame é uma tabela onde cada coluna é uma Series que compartilha o mesmo índice.

```
# Criando DataFrame a partir de dicionário
dados = {
    "nome"      : ["Ana", "Bruno", "Carla", "Diego"],
    "idade"     : [22, 25, 21, 28],
    "nota"      : [8.5, 7.0, 9.2, 6.3],
    "aprovado"  : [True, True, True, False],
}
df = pd.DataFrame(dados)

# Visualizando
print(df)          # tabela completa
```

```

print(df.head(2)) # primeiras 2 linhas
print(df.tail(2)) # últimas 2 linhas
print(df.shape)   # (4, 4)
print(df.dtypes)  # tipos de cada coluna
print(df.info())  # resumo estrutural

```

5.4 Seleção e Filtragem de Dados

```

# Selecionar uma coluna → Series
print(df["nome"])

# Selecionar múltiplas colunas → DataFrame
print(df[["nome", "nota"]])

# loc – seleção por rótulo de linha e coluna
print(df.loc[0, "nome"]) # "Ana"
print(df.loc[1:2, ["nome", "nota"]]) # linhas 1-2, colunas nome e nota

# iloc – seleção por posição inteira
print(df.iloc[0, 0]) # "Ana"
print(df.iloc[:2, :3]) # primeiras 2 linhas, 3 colunas

# Filtro booleano
aprovados = df[df["aprovado"] == True]
acima8 = df[df["nota"] >= 8.0]
jovens_ap = df[(df["idade"] < 25) & (df["aprovado"])]

```

5.5 Operações e Transformações

```

# Adicionar nova coluna calculada
df["nota_ponderada"] = df["nota"] * 1.1
df["conceito"] = df["nota"].apply(
    lambda n: "A" if n >= 9 else ("B" if n >= 7 else "C")
)

# Renomear colunas
df.rename(columns={"nota": "media"}, inplace=True)

# Ordenar
df_ord = df.sort_values("media", ascending=False)

# Agrupar e agregar (groupby)
resumo = df.groupby("aprovado")["media"].agg(["mean", "min", "max"])
print(resumo)

# Estatísticas descritivas de colunas numéricas
print(df.describe())

```

5.6 Leitura e Escrita de Arquivos

O Pandas suporta CSV, Excel, JSON, SQL e vários outros formatos:

```

# Leitura de CSV
df = pd.read_csv("dados.csv", sep=";", encoding="utf-8")

# Leitura de Excel
df = pd.read_excel("planilha.xlsx", sheet_name="Plan1")

# Escrita

```

```
df.to_csv("saida.csv", index=False)    # sem coluna de índice
df.to_excel("saida.xlsx", index=False)

# Pré-visualização após leitura
print(df.shape)      # linhas × colunas
print(df.head())     # primeiras 5 linhas
print(df.isnull().sum()) # valores ausentes por coluna
```

5.7 Tratamento de Dados Ausentes

```
import numpy as np

df2 = pd.DataFrame({
    "x": [1, None, 3, None, 5],
    "y": [10, 20, None, 40, 50],
})

# Verificar
print(df2.isnull())      # máscara booleana
print(df2.isnull().sum()) # contagem por coluna

# Remover linhas com qualquer NaN
df_limpo = df2.dropna()

# Preencher NaN com um valor
df_fill1 = df2.fillna(0)
df_fill2 = df2.fillna(df2.mean()) # preenche com média da coluna
```

5.8 Exemplo Prático — Análise de Desempenho de Turma

Cenário: o professor recebe um CSV com notas de três bimestres e precisa calcular a média final, o conceito e gerar um relatório.

```
import pandas as pd

# Simula um CSV já lido
df = pd.DataFrame({
    "aluno" : ["Ana", "Bruno", "Carla", "Diego", "Elena"],
    "b1"    : [8.0, 6.5, 9.5, 5.0, 7.0],
    "b2"    : [7.5, 7.0, 8.5, 6.0, 8.0],
    "b3"    : [9.0, 5.5, 9.0, 4.5, 8.5],
})

# Calcular média final (pesos iguais)
df["media"] = df[["b1", "b2", "b3"]].mean(axis=1).round(2)

# Atribuir conceito
def conceito(m):
    if m >= 9.0: return "A"
    if m >= 7.0: return "B"
    if m >= 6.0: return "C"
    return "D (Reprovado)"

df["conceito"] = df["media"].apply(conceito)

# Relatório resumido
print(df[["aluno", "media", "conceito"]].to_string(index=False))
print()
print(f"Média da turma : {df["media"].mean():.2f}")
print(f"Aprovados      : {(df["media"] >= 6).sum()} / {len(df)}")
```

```
print(f"Melhor aluno    : {df.loc[df["media"].idxmax(), "aluno"]}")

# Exportar
df.to_csv("relatorio_turma.csv", index=False)
```

6. Referências e Recursos

Para aprofundamento nos temas desta aula, consulte:

- **Documentação oficial Python:** <https://docs.python.org/3/>
- **NumPy User Guide:** <https://numpy.org/doc/stable/user/>
- **Python Tutor** (visualizador de código): <https://pythontutor.com>
- **Livro:** *Python Crash Course* — Eric Matthes (No Starch Press)
- **Livro:** *Python para Análise de Dados* — Wes McKinney (O'Reilly)
- **Documentação oficial Pandas:** <https://pandas.pydata.org/docs/>
- **10 Minutes to Pandas:** https://pandas.pydata.org/docs/user_guide/10min.html